Motor de búsqueda de funciones similares vía AST

Informe técnico breve | Trabajo práctico grupal 2026

Integrantes: Ayala Nereo · Bertone Zoe · Stoffel Francisco

1. Objetivo y fundamento

Se desarrolló una herramienta para localizar funciones estructuralmente similares dentro de un repositorio local de archivos Python. En lugar de comparar el texto fuente, se analiza el Árbol de Sintaxis Abstracta (AST) obtenido por el parser de Python. Este árbol representa las construcciones sintácticas del programa y permite reconocer semejanzas aun cuando cambian los nombres de variables, parámetros o literales.

2. Decisiones de diseño

Arquitectura modular. La solución se separó en extractor, normalizador, serializador, módulo de similitud y buscador, más una interfaz de línea de comandos (CLI). Esta división mantiene responsabilidades independientes y facilita las pruebas con pytest.

Extracción. El módulo extractor.py recorre recursivamente los archivos .py con pathlib, los analiza mediante ast.parse() y obtiene FunctionDef y AsyncFunctionDef, incluidos métodos y funciones anidadas. De cada función conserva nombre, archivo, línea y subárbol AST. Si un archivo presenta errores de lectura o sintaxis, se informa y se continúa con los restantes.

Normalización. normalizador.py utiliza copy.deepcopy() y ast.NodeTransformer para no alterar el árbol original. Renombra cada función raíz como FUNC y sustituye identificadores por VAR1, VAR2, etc., manteniendo su correspondencia dentro de la función. Los literales numéricos y de texto pasan a LIT; True, False y None se conservan. Se preservan operadores y nodos de control, que aportan información estructural relevante.

Serialización. serializador.py recorre el AST normalizado en pre-order (nodo padre y luego hijos con ast.iter_child_nodes()) y genera una lista determinista de tokens, por ejemplo BinOp, Add, Name:VAR1 o Constant:LIT. No incorpora nombres originales de funciones ni ubicaciones del archivo.

Búsqueda e interfaz. buscador.py prepara una sola vez los tokens de cada función, identifica la consulta mediante archivo.py:nombre, excluye la propia función y ordena las candidatas por puntuación descendente. buscar_similares.py expone una CLI con --dir, --funcion, --top, --n y --umbral. El ranking Top-K incluye score y clasificación de posible duplicado.

3. Métrica de similitud y justificación

Se eligió el índice de Jaccard aplicado a conjuntos de n-gramas consecutivos de tokens. Dados los conjuntos A y B, la similitud es $J(A,B) = |A \cap B| / |A \cup B|$ y toma valores entre 0 (sin n-gramas compartidos) y 1 (conjuntos iguales). El uso de n-gramas conserva parte del contexto y del orden local que se perdería comparando únicamente tokens individuales.

Se emplean trigramas (n = 3) por defecto: ofrecen un compromiso entre tolerancia a cambios menores y capacidad de distinguir construcciones diferentes, frente a bigramas más permisivos o n-gramas mayores más sensibles. El umbral configurable se fijó en 0,75: una puntuación igual o superior marca la función como «candidata a duplicado», sin demostrar equivalencia de comportamiento. La búsqueda de una función realiza M − 1 comparaciones para M funciones extraídas; no requiere infraestructura de indexación para el tamaño previsto del repositorio.

4. Limitaciones conocidas: clones que puede no detectar

Clones con cambios extensos de orden o estructura (tipo 3). La reordenación de bloques, inserciones, eliminaciones o fragmentación de instrucciones altera los trigramas. Aunque algunos cambios pequeños se toleran, clones modificados pueden quedar por debajo del umbral. Además, los identificadores VARx se asignan en orden de aparición, por lo que un reordenamiento puede cambiar también su numeración.

Clones semánticos (tipo 4). Dos funciones que resuelven el mismo problema con algoritmos diferentes —por ejemplo, un cálculo iterativo frente a uno recursivo— pueden tener AST muy distintos y no ser reconocidas como similares. El motor mide estructura sintáctica, no equivalencia funcional.

Renombrados y ámbitos complejos. La normalización no construye una tabla de símbolos ni resuelve completamente los alcances de Python: funciones anidadas comparten el mapa de identificadores de su raíz y nombres iguales en ámbitos diferentes pueden confundirse. También se conservan nombres de atributos y etiquetas de argumentos por palabra clave, por lo que ciertos renombrados no quedan neutralizados.

Pérdida de detalle y falsos positivos. Al convertir el AST a pre-order no se codifican explícitamente los límites de cada subárbol. Jaccard emplea conjuntos: ignora cuántas veces se repite un n-grama. A su vez, reemplazar números y textos diferentes por LIT puede elevar artificialmente el score de funciones con comportamientos distintos. Los resultados requieren revisión humana.

Alcance acotado. La comparación se limita a funciones de archivos .py. No identifica duplicados de clases completas, similitudes repartidas entre varias funciones ni clones de otros lenguajes.

5. Posibles mejoras

Mayor precisión en la normalización. Incorporar análisis de ámbitos y símbolos (local, global y nonlocal), y distinguir identificadores de variables, funciones externas y nombres de atributos. También podría preservarse la categoría de los literales para reducir coincidencias engañosas.

Métricas complementarias. Comparar Jaccard con Levenshtein normalizada o con medidas basadas directamente en árboles (por ejemplo, distancia de edición de árboles), para considerar orden, frecuencia de patrones y jerarquía estructural.

Detección de clones parciales. Analizar bloques o subárboles de las funciones, en lugar de comparar únicamente la función completa, para identificar fragmentos reutilizados con cambios y desplazamientos.

Evaluación y escalabilidad. Construir un conjunto etiquetado más amplio de clones y no clones, medir precisión y exhaustividad, ajustar n y umbral empíricamente y, para repositorios grandes, incorporar índices o caché de n-gramas.

6. Verificación y conclusión

El repositorio incorpora pruebas con pytest para los módulos y cuatro casos de integración requeridos por la cátedra: dos positivos (renombrado de variables y reordenamiento de instrucciones) y dos negativos (estructuras y algoritmos diferentes). La solución cumple el propósito de una búsqueda estructural básica y extensible; las coincidencias se presentan como indicios de duplicación, nunca como prueba de identidad semántica.